

ROBOTICS SOFTWARE — TECHNICAL DEEP DIVE

Autonomous Farm-Lane Navigation & Plant Counting with a Single RGB-D Camera

Depth-geometric perception · Nav2 · RTAB-Map · ROS 2 Humble

Husky A200

1× RealSense D435

Nav2 (A* + DWB)

RTAB-Map SLAM

0 learned models

Interview presentation — architecture, algorithms, config & reproducibility

Agenda

1 Introduction & problem

2 Evaluation metrics

3 Core design idea

4 System architecture

5 Perception vs. SLAM split

6 Camera intrinsics & calibration

7 RGB-D fusion & TF frames

8 Perception pipeline

9 RANSAC ground plane

10 Obstacle detection

11 Lane detection (bed rows)

12 Plant detection & counting

13 De-duplication logic

14 Localization overview

15 RTAB-Map visual SLAM

16 Nav2 stack & Behavior Tree

17 How A* / NavFn works

18 How DWB works

19 How costmaps work

20 Nav2 config guide

21 Hardcoded vs learned · Q&A

Introduction — the task & its hard constraints

- ▶ A Husky UGV must drive itself from a BLUE start marker to a GREEN goal marker through a Gazebo farm of irregular, non-parallel crop lanes...
- ▶ ...while detecting and counting the plants it passes.
- ▶ Four constraints drive every design decision:
 - ONE sensor — a single RealSense RGB-D camera. No LiDAR / 2nd camera / GPS / localization sensor. Wheel odometry (proprioceptive) is allowed.
 - NO colour — HSV / RGB thresholding / green-pixel following = automatic disqualification. Perception must be geometric.
 - NO hardcoded route — real-time planning with ROS 2 Humble + Nav2.
 - Runtime start/goal — marker poses read from the world live, never literals in code.

Why these constraints matter

Single low camera

robot discovers the field incrementally — cannot pre-know lanes

No colour

must infer structure from 3-D geometry (depth), not appearance

No fixed route

navigation must react to what is perceived, frame by frame

Runtime markers

start/goal recovered from Gazebo entity states at launch

How the system is scored — four metrics

01**Navigation success**

Reaches the green goal without a navigation failure.

02**Lane coverage**

Covered lane area $\div$ total lane area $\times$ 100. Higher is better.

03**Path efficiency**

Distance driven + deviation between planned and actual path.

04**Count accuracy**

Estimated vs ground truth — false / missed / duplicate.

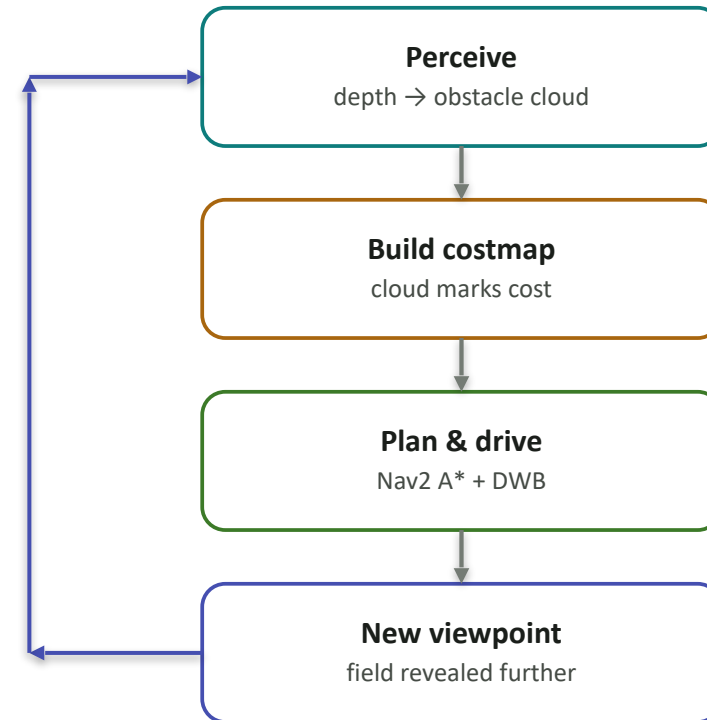
DESIGN TARGET

The mission sweeps the field lane-by-lane (coverage) instead of dashing straight to green — directly optimising metric 2 while still terminating at the goal. Plant counting is de-duplicated in a world frame so re-observations never inflate metric 4.

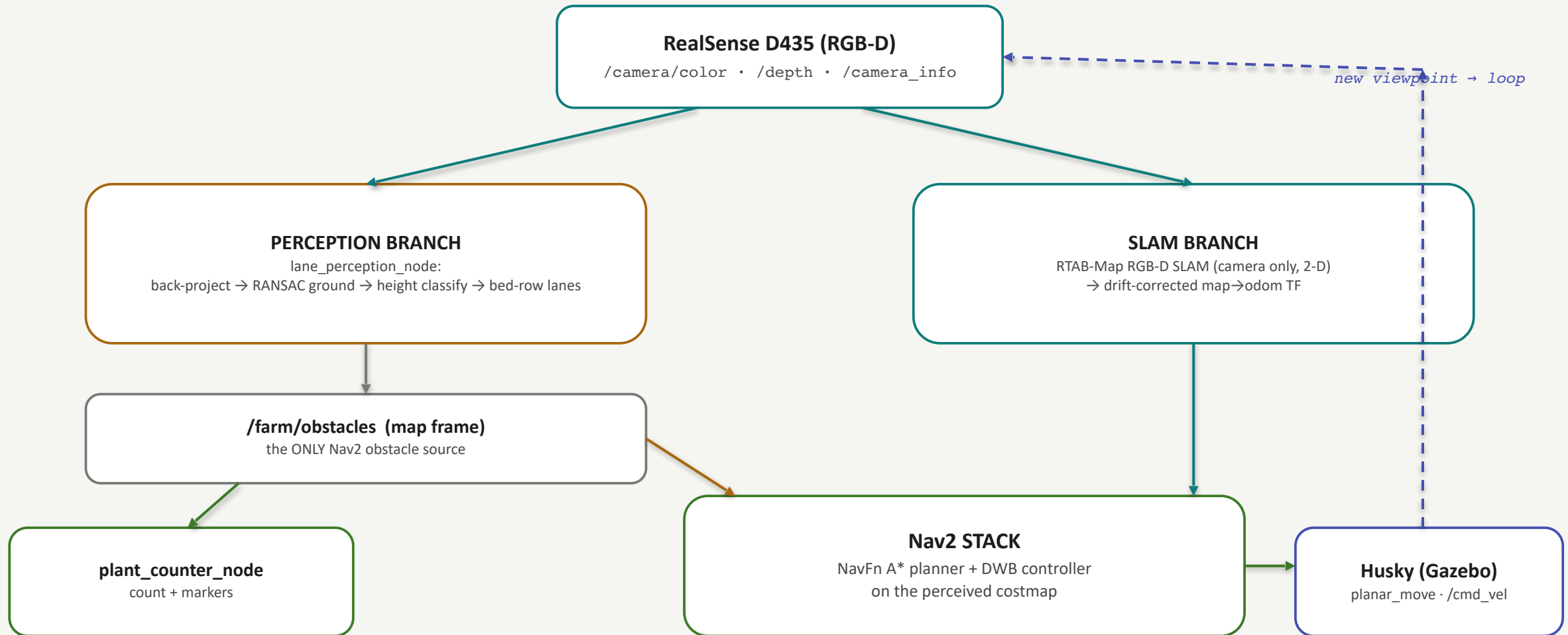
Core design idea — perception feeds the costmap, not the wheels

- ▶ A robot cannot FOLLOW a lane it has not yet seen — and a hardcoded route is banned.
- ▶ So lane perception is NEVER a steering signal.
- ▶ Instead:
 - Depth geometry → an obstacle point cloud.
 - That cloud IS the Nav2 costmap (only obstacle source).
 - Nav2 plans + re-plans a collision-free path to the runtime goal.
 - Motion → new viewpoint → costmap densifies → re-plan. Loop.
- ▶ Result: fully perception-driven, real-time, no fixed trajectory anywhere in code.

The perception→plan→move loop



System architecture — dual-branch data flow



Why perception and SLAM are kept separate

Perception estimates STRUCTURE

- ▶ Answers: “where are the beds / plants / free space?”
- ▶ Output: obstacle cloud + lane centres, per frame.
- ▶ Depends only on depth geometry.
- ▶ Swappable: a learned segmentation head could replace it behind the same /farm/obstacles topic.

SLAM estimates POSE

- ▶ Answers: “where is the robot in the world?”
- ▶ Output: drift-corrected map → odom transform.
- ▶ Independent of the obstacle pipeline.
- ▶ Swappable: any localization (SLAM, AMCL, GPS) can be dropped in without touching perception.

MODULARITY

Decoupling structure-estimation from pose-estimation is the key architectural choice — each can be replaced or upgraded independently.

Camera intrinsics & calibration

Intrinsics (in-sensor)

- ▶ Horizontal FOV 1.211 rad ($\approx 69^\circ$); 640×480 @ 15 Hz.
- ▶ Clip range 0.10 – 10 m; Gaussian depth noise $\sigma = 0.005$ m.
- ▶ Gazebo derives f_x , f_y , c_x , c_y and publishes them on `/camera/depth/camera_info` — read LIVE, never hardcoded.

Back-projection (pixel $\rightarrow$ 3-D)

For each depth pixel (u, v, z):

$$x = (u - c_x) \cdot z / f_x$$

$$y = (v - c_y) \cdot z / f_y$$

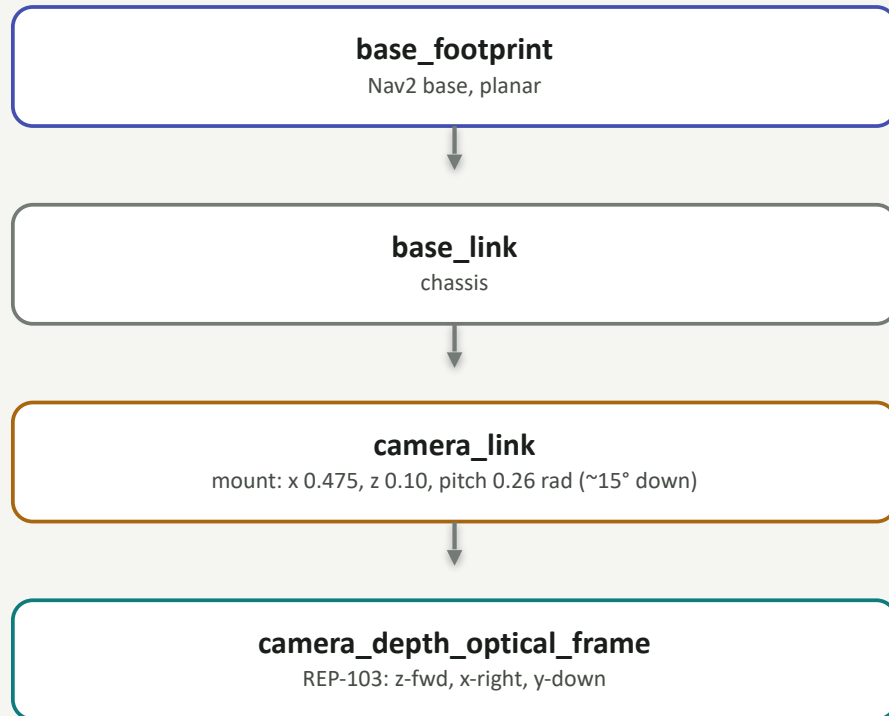
$$z = z$$

$\rightarrow$ organized metric point cloud in the camera optical frame.

INTERVIEW POINT — WHY INTRINSICS MATTER

Intrinsics convert raw pixels + a depth value into metric 3-D. Without correct $f_x/f_y/c_x/c_y$ the cloud is geometrically wrong and RANSAC / height thresholds break. In sim they are exact and published on `camera_info`; on real hardware they come from factory or checkerboard calibration.

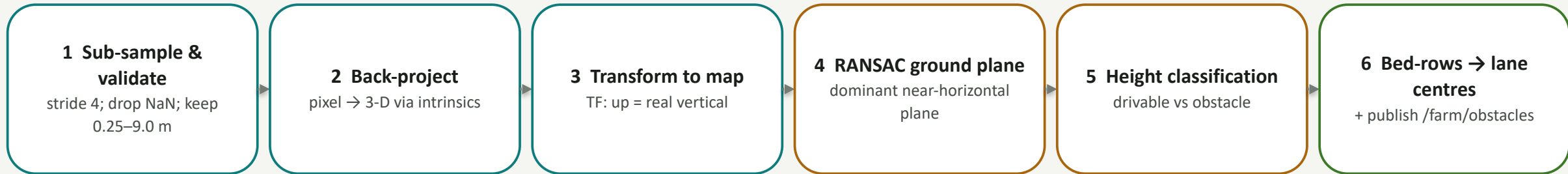
RGB-D fusion & the extrinsic frame chain



How RGB and depth combine

- ▶ Exinsics (URDF joints → TF, via robot_state_publisher) place the metric cloud in a gravity-aligned body/world frame.
- ▶ This is the calibration that makes “height above ground” physically meaningful.
- ▶ **Depth does ALL decision-making.**
- ▶ RGB is only the canvas for the overlay video — every classified 3-D point carries its source pixel (u,v), so labels can be painted back onto the colour image.
- ▶ **No perception decision ever reads a colour value → the no-HSV rule cannot be violated by construction.**

Lane / free-space perception — pipeline



RUNS AT ~6 HZ, PURE PYTHON + NUMPY

Everything is classical geometry — deterministic, dataset-free, auditable. Each stage below is expanded on the next slides.

- ▶ Synchronised (depth, camera_info, color) triple via ApproximateTimeSynchronizer (0.10 s slop) — RGB carried only for the overlay.

RANSAC ground-plane estimation — how & why

Algorithm (60 iterations)

1. Sample 3 random points $\rightarrow$ candidate plane $n \cdot p + d = 0$

2. **REJECT if $|n \cdot \hat{z}| < 0.85$ (keep near-horizontal only)**

3. Count inliers: $|n \cdot p + d| < 0.03 \text{ m}$

4. Keep the plane with the most inliers

5. Fallback to flat $z = 0$ if none found

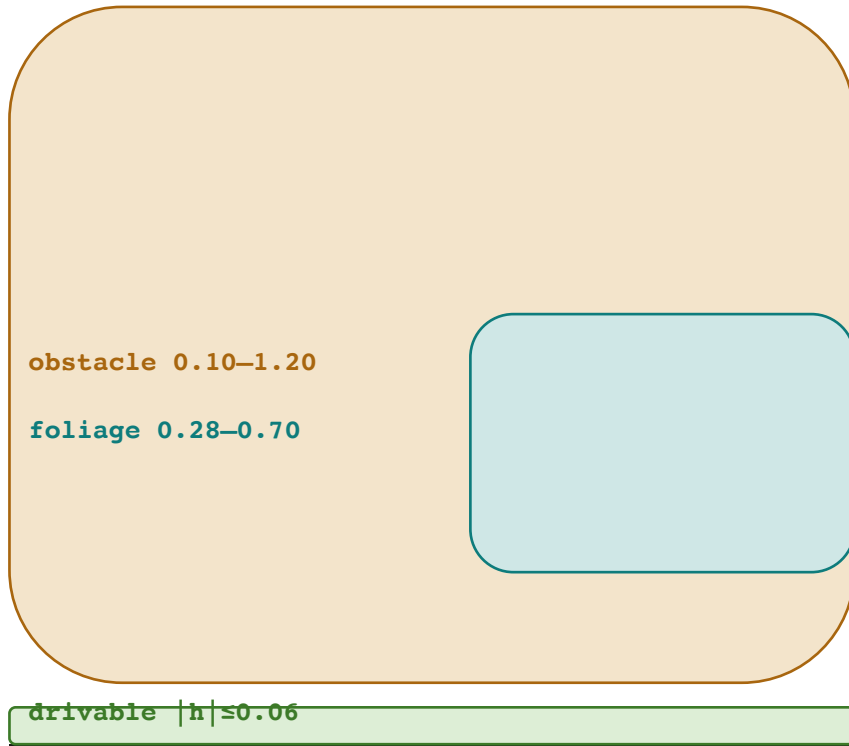
Points pre-filtered to the low band ($\text{map-}z < 0.4 \text{ m}$) so the floor dominates the sample.

Why RANSAC (design choice)

- ▶ Does NOT assume ground = $z = 0 \rightarrow$ robust to camera tilt / mount error / sloped terrain (classifies by signed height above the FITTED plane).
- ▶ $|n \cdot \hat{z}| > 0.85$ gate: beds & plants are outliers and can never win the fit — only the broad farm floor can.
- ▶ **Orientation-invariant $\rightarrow$ satisfies FR-1 (robust to lane angle, geometry, appearance): a lane at any heading gives the same height signature.**
- ▶ Cheap: 60 iterations on a sub-sampled cloud runs in real time in pure Python.

Obstacle detection — height classification

Signed height $h = n \cdot p + d$



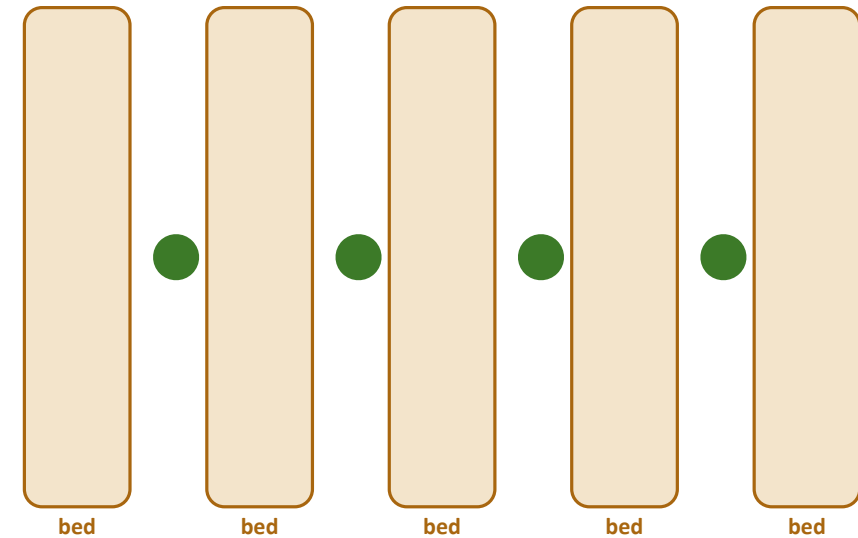
- ▶ Each point classified by SIGNED height above the fitted plane:
 - $|h| \leq 0.06$ m $\rightarrow$ drivable lane surface (GROUND_BAND).
 - $0.10 < h < 1.20$ m $\rightarrow$ obstacle: crop beds (~0.10 m), plants, debris.
- ▶ Obstacle points $\rightarrow$ /farm/obstacles as a PointCloud2 in the map frame.
- ▶ This is the **SOLE** input to Nav2's obstacle layer — no colour, no LiDAR.
- ▶ Robot-clear radius drops points too close to the base so it never traps itself in the costmap.
- ▶ Bed height ≈ 0.10 m and canopy ≈ 0.42 m are the physical facts every threshold is derived from — not fitted to data.

Lane detection — bed rows → lane centres

- ▶ Within a forward window, obstacle points are projected onto the LATERAL (y) axis.
- ▶ A 1-D histogram of that projection reveals contiguous peaks = crop-bed rows.
- ▶ **The free-space MIDPOINT between two adjacent rows = a lane centre.**
- ▶ The nearest corridor becomes a look-ahead centreline waypoint (/farm/lane_center), drawn in RViz.
- ▶ **Recomputed every frame → handles irregular, non-parallel, varying-width lanes by construction.**
- ▶ A gap whose bed-spacing is below the pass threshold (the 0.4 m narrow lane) is excluded as non-traversable.

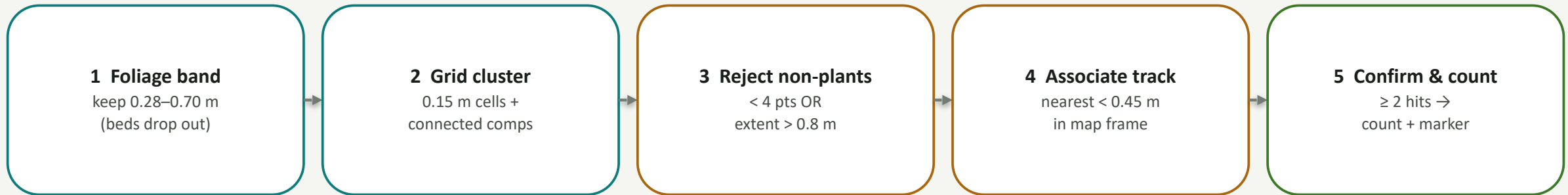
Lateral histogram → lanes

obstacle count vs lateral offset y →



● = lane centre (row midpoint)

Plant detection & counting — pipeline



- ▶ Subscribes to the SAME obstacle cloud — ground & free space already removed, so only beds + plants remain.
- ▶ Foliage-height filter keeps the canopy (~0.42 m centre) and removes the 0.10 m beds — pure geometry, no colour.
- ▶ Elongation guard (extent > 0.8 m) rejects 10 m bed-wall fragments → essentially no false positives.
- ▶ Ground truth this world: 25 upright (5 beds × 5) + 1 fallen = 26. Fallen plant sits at bed height → known hard miss.

De-duplication — counting each plant ONCE

- ▶ Every cluster centroid is an **OBSERVATION** in the drift-corrected map frame.
- ▶ Because that frame is world-stable, the same physical plant lands at ~same (x, y) from every viewpoint.
- ▶ **Association: match to nearest existing track within 0.45 m →**
 - match found → UPDATE that track (running average, weight $1/(\text{obs}+1)$).
 - none → create a NEW track.
- ▶ **Confirmation: a track is counted only after ≥ 2 observations (kills single-frame flicker).**
- ▶ **Seeing a plant 20× → still one track, counted once.**

Why $r = 0.45$ m

- ▶ Larger than residual inter-viewpoint drift → the SAME plant re-associates.
- ▶ Smaller than the minimum spacing between DIFFERENT plants → neighbours never merge.
- ▶ **It sits in the sweet spot between those two scales.**
- ▶ Tuning up $\Rightarrow$ merge risk; tuning down $\Rightarrow$ duplicate risk.
- ▶ **Entirely geometric — no learned model, no appearance.**

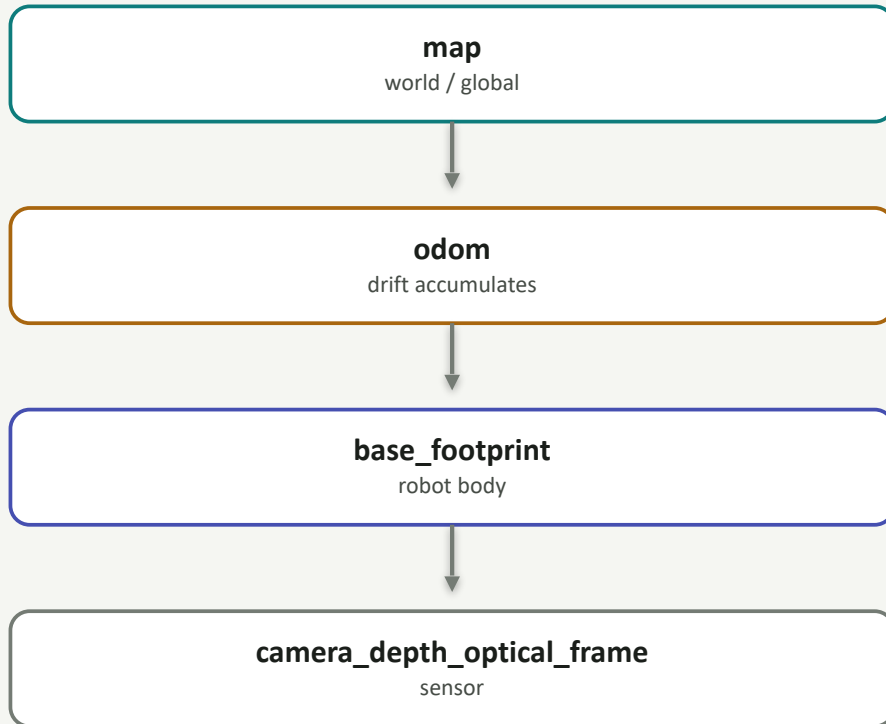
Localization — odometry + a world-aligned frame

- ▶ **NO AMCL.** AMCL estimates pose from LASER scans — we have no LiDAR, only a depth camera.
- ▶ odom → base_footprint: from the planar_move drive plugin (wheel/model odometry). Smooth, high-rate, but drifts.
- ▶ **map → odom: drift correction. Two options:**
 - RTAB-Map visual SLAM (default) — refines odom with the camera.
 - Static identity transform (use_slam:=false) — map = odom = world, since sim odom is already world-aligned & accurate.
- ▶ **Nav2 plans in map; control executes in odom.**

Why no AMCL — talking point

- ▶ AMCL = particle filter over a KNOWN occupancy grid using laser scans.
- ▶ No laser → AMCL cannot run.
- ▶ In simulation the odometry transform is accurate and world-aligned, so a static map→odom is sufficient and stable.
- ▶ **Visual SLAM (RTAB-Map) is the camera-only upgrade that corrects real drift.**

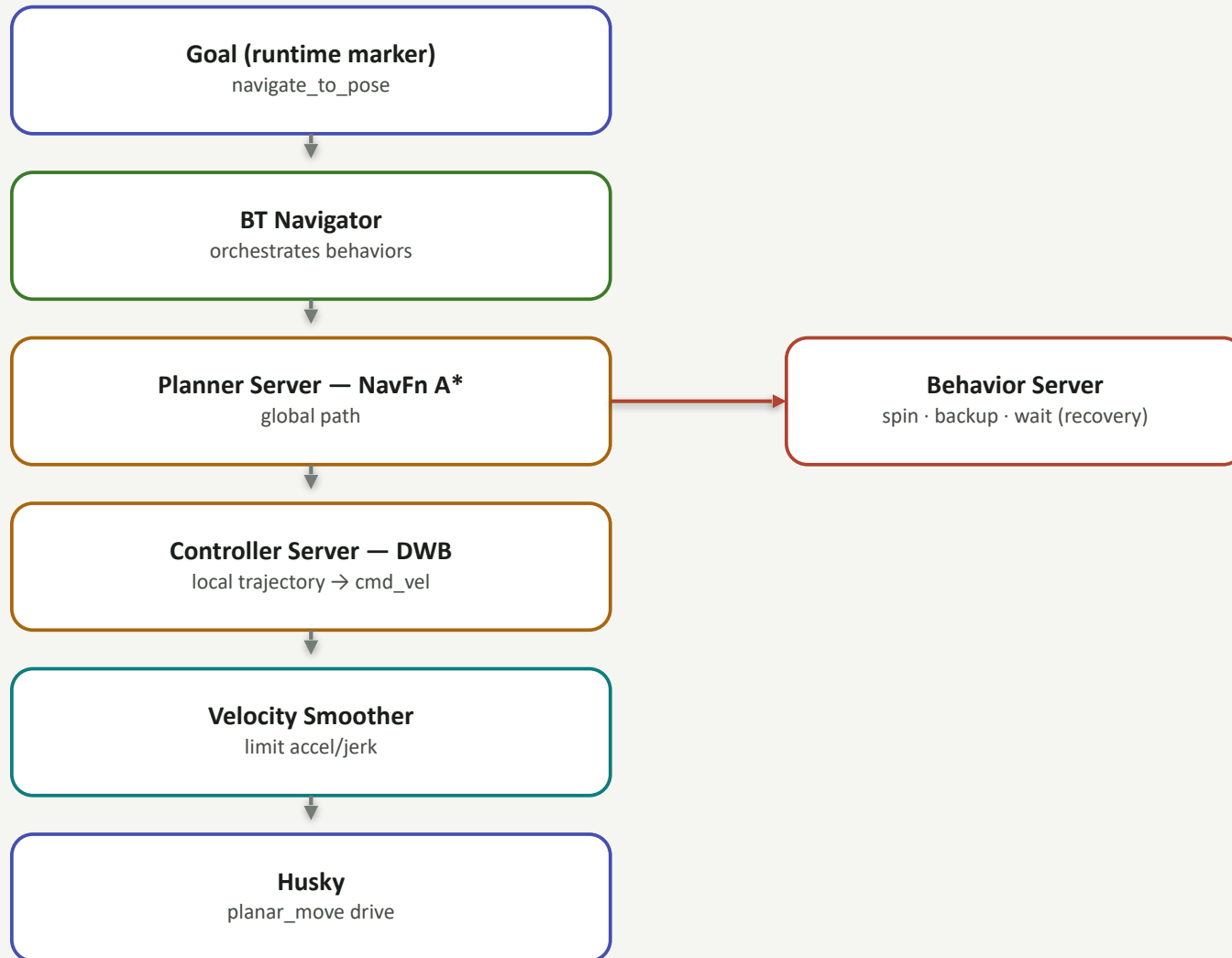
Visual SLAM — how RTAB-Map is used



How & why

- ▶ RGB-D SLAM on the RealSense ALONE. Wheel odom is the motion prior; RTAB-Map refines it with classical visual feature registration + loop closure.
- ▶ Constrained to 2-D (Reg/Force3DoF, Optimizer/Slam2D) — the farm is planar → far more robust on the low-texture floor.
- ▶ Reg/Strategy 0 = visual (classical features, NOT a trained network). Vis/MinInliers 10.
- ▶ Grid/FromDepth = false — the costmap comes from OUR obstacle cloud, not RTAB's grid.
- ▶ Map origin anchored at the spawn pose → goal_sender converts world coords into map at runtime.
- ▶ Limitation: on feature-poor floor, loop closure fires rarely → leans on wheel odom. Depth-ICP would be the next upgrade.

Nav2 stack & the Behavior Tree



Why a Behavior Tree?

- ▶ Each behavior — plan, follow path, recover — is a NODE in a tree, not code in a monolithic state machine.
- ▶ Modular & easy to extend: add/reorder recovery behaviors without rewriting logic.
- ▶ BT ticks at ~10 Hz; on planner/controller failure it falls through to Behavior Server recovery (spin, back-up, wait), then retries.
- ▶ **Clean separation: planning ≠ control ≠ recovery.**

How the global planner works — A* / NavFn

A* search — the idea

- ▶ Searches the global costmap grid for the lowest-cost path from robot to goal.
- ▶ **Expands cells by priority $f(n) = g(n) + h(n)$:**
 - $g(n)$ = cost so far (distance + costmap cost).
 - $h(n)$ = heuristic (straight-line to goal) — guides search toward the goal.
- ▶ Admissible heuristic → guaranteed shortest path, far fewer expansions than Dijkstra.
- ▶ NavFn = Nav2's wavefront/A* planner over the 2-D costmap.

How it's used here

- ▶ Plugin nav2_navfn_planner/NavfnPlanner, use_astar: true, tolerance 0.5 m.
- ▶ allow_unknown: true → plans INTO not-yet-seen space and refines as the camera reveals it.
- ▶ Runs on a rolling global costmap large enough to cover the whole farm.
- ▶ Obstacle layer marks the perceived beds (clearing off) → routes AROUND discovered dead-ends instead of re-entering.
- ▶ **Re-plans continuously (~2 Hz) as new obstacles appear.**

How the local controller works — DWB

DWB = Dynamic Window + critics

- ▶ Each control cycle (20 Hz) it SAMPLES many (v, ω) velocity commands within the robot's dynamic limits (the “window”).
- ▶ Each sample is forward-SIMULATED into a short trajectory.
- ▶ **Every trajectory is SCORED by a weighted sum of CRITICS:**
 - PathDist / PathAlign — stay on & aligned to the global path.
 - GoalDist / GoalAlign — make progress to the goal.
 - BaseObstacle — penalise collisions; Oscillation — anti-jitter.
- ▶ **Best-scoring trajectory's first command** → `/cmd_vel`.

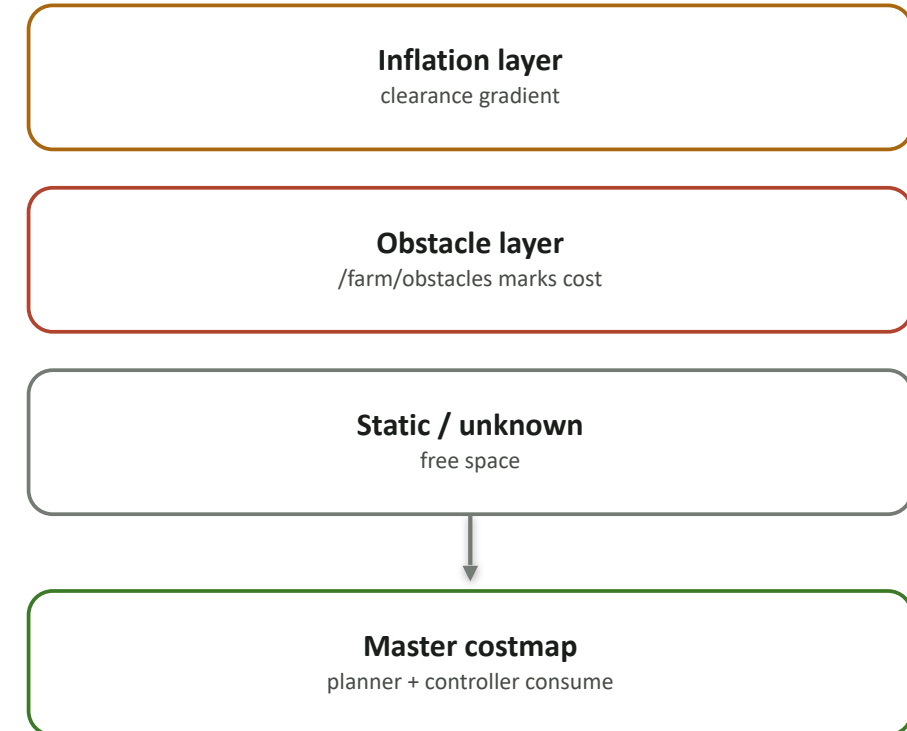
How it's tuned here

- ▶ **Heavy PathDist / PathAlign weights** → the controller HUGS the global path, keeping the 0.67 m skid base centred in ~1 m lanes.
- ▶ $\text{max_vel_x} \approx 0.24$, $\text{max_vel_theta} \approx 0.8$ → stable in tight lanes.
- ▶ `yaw_goal_tolerance` wide open — a skid base can't hit an exact final heading, else the goal aborts just short.
- ▶ Works on the depth-built costmap (which inflates the beds) → far more robust than open-loop lane following.

How the costmaps work

- ▶ A costmap is a 2-D grid (0.05 m cells) where each cell holds a traversal cost 0–254.
- ▶ Built from **LAYERS** composited together:
 - Obstacle layer — marks cells hit by /farm/obstacles points (marking on, clearing off).
 - Inflation layer — spreads a decaying cost around each obstacle so paths keep clearance.
- ▶ GLOBAL costmap: whole-field, rolling → NavFn plans on it.
- ▶ LOCAL costmap: small 6×6 m rolling window around the robot → DWB avoids obstacles on it.
- ▶ Inflation radius 0.36 m $\geq$ inscribed radius (0.35) → path stays off beds; gentle cost_scaling keeps the lane centre attractive.

Layered composition



Nav2 configuration guide (nav2_params.yaml)

Server	Key params	Value / choice
bt_navigator	global_frame / base	map / base_footprint
planner_server	NavFn A*	use_astar, tol 0.5, allow_unknown
controller_server	DWB critics	PathDist/PathAlign high; max_vel_x 0.24, θ 0.8
goal_checker	xy / yaw tol	0.25–0.35 m / open (3.2 rad)
local_costmap	6×6 m, 0.05	obstacle + inflation, rolling
global_costmap	whole field, 0.05	obstacle (clearing off) + inflation
inflation	radius / scaling	0.36 m / 2.8–3.0
obstacle_layer	source	/farm/obstacles, marking on, clearing off
velocity_smoother	accel limits	smooth cmd_vel before drive

Hardcoded vs learned · interview Q&A

Hand-engineered (all of it)

- ▶ RANSAC ground fit
- ▶ Height-threshold classify
- ▶ Histogram bed-row lanes
- ▶ Grid clustering + tracks
- ▶ RTAB-Map classical features
- ▶ NavFn A* + DWB
- ▶ ~50 constants, 0 learned

Learned — NONE

- ▶ No HSV / RGB thresholds
- ▶ No trained network
- ▶ No colour segmentation
- ▶ Drop-in later: seg-head behind /farm/obstacles or a detector behind plant tracks — same interface.

Likely questions

Q: Why not AMCL?

A: No laser; sim odom already world-aligned.

Q: Why RANSAC not $z=0$?

A: Robust to tilt/slope; beds are outliers.

Q: Avoid double counts?

A: Track association in map frame + ≥ 2 hits.

Q: Why DWB over pure-pursuit?

A: Path-hugging critics keep skid base centred.

Q: Perception steers?

A: No — it feeds the costmap; Nav2 steers.

Reproduce & key commands

```
# build image (first run) + launch, headless
cd farm_nav_ws
./run.sh                # GUI (Gazebo + RViz)
./run.sh headless:=true use_rviz:=false

# native (Humble on 22.04)
colcon build --symlink-install
source install/setup.bash
ros2 launch farm_bringup bringup.launch.py

# inspect a live run
ros2 topic echo /farm/plant_count
ros2 topic echo /mission/status
```

KEY FACTS

- ▶ Launch args: use_rviz · use_slam · headless · spawn_x/y/yaw
- ▶ Deliverables → ./output : run_video.mp4, path_overlay.png
- ▶ Ground truth: 26 plants · 4 drivable lanes (0.4 m lane skipped)
- ▶ Start $\approx (5.8, -3.0)$ · Goal $\approx (-5.8, 2.7)$, read at runtime
- ▶ GPU strongly preferred (software GL times out Nav2 lifecycle)

Depth-geometric · real-time Nav2 · camera-only SLAM · zero learned models — explainable, auditable, reproducible.